PitchShift Audio Engine

Technical Architecture, Digital Signal Processing (DSP) Pipeline, & Implementation Documentation

This document provides a comprehensive technical breakdown of the **PitchShift Chrome Extension**. It details the underlying mathematical concepts, the Digital Signal Processing (DSP) algorithms, and the Web Audio API architecture used to achieve real-time, high-fidelity audio transposition and Music Information Retrieval (MIR) within a browser environment.

1. Core Audio Pipeline Architecture

Modern web applications (like Spotify and YouTube) use isolated DOM structures and secure audio contexts to play media. Standard pitch-shifting via HTML5 ***playbackRate*** causes unacceptable "chipmunk" artifacts and alters the tempo of the track. To solve this, PitchShift implements a custom DSP pipeline.

> **The AudioWorklet Paradigm**
>
> Because audio processing requires strict timing, doing heavy math on the browser's main JavaScript thread causes stuttering and UI freezes. PitchShift utilizes the `AudioWorkletProcessor` API. This runs the DSP code in a dedicated, high-priority background thread completely isolated from the webpage's visual rendering.

The routing pipeline operates as follows:

- **Media Interception:** The extension proxies `AudioContext.prototype.createMediaElementSource` to intercept the raw audio stream before it reaches the speakers.
- **Graceful Fallback:** For sites not using Web Audio API natively, a recursive DOM scanner finds `<audio>` and `<video>` elements and hooks them manually, applying CORS headers (`crossOrigin="anonymous"`) to prevent security blocks.
- **True Bypass:** When the shift is ***0.0*** semitones, the engine enters a zero-latency state, writing input buffers directly to output buffers to preserve 100% of the original audio quality while running a lightweight analysis thread in the background.

2. The Phase Vocoder Algorithm

The core of the pitch-shifting engine is a **Phase Vocoder**. Unlike simple time-domain stretching, a Phase Vocoder translates the audio into the frequency domain, manipulates the frequencies, and translates it back. This allows pitch shifting *without* altering the tempo.

1. STFT (Short-Time Fourier Transform)

The audio stream is sliced into overlapping windows (4096 samples per frame, overlapping by 75%). Each frame is multiplied by a Hann Window to prevent spectral leakage, and converted into the frequency domain via a Fast Fourier Transform (FFT).

2. Phase Unwrapping

Because the FFT only provides phase values mapped between $-\pi$ and $+\pi$, the engine calculates the phase difference between consecutive frames. This "unwraps" the phase, allowing the exact, true frequency of a bin to be determined.

True Frequency: $\Delta\varphi = \varphi_{current} - \varphi_{previous} - ExpectedPhaseAdvance$

$$f_{true} = BinFrequency + (\Delta\varphi \times {SampleRate}/{2\pi H})$$

The true frequencies are then multiplied by the Pitch Factor ($pf = 2^{(semitones/12)}$), and the audio is synthesized back into the time domain using an Inverse FFT (IFFT) and Overlap-Add methodology.

3. High-Fidelity Enhancements (Pro DSP)

A standard Phase Vocoder produces "watery" or "metallic" artifacts. PitchShift implements several advanced proprietary techniques to clean the signal.

Stereo Phase-Locking (Master/Slave Architecture)

A song's stereo width is defined by microscopic phase differences between the Left and Right channels. If a Phase Vocoder processes Left and Right independently, it alters their phases differently, destroying the stereo image (Phase Smearing) and collapsing the audio to mono.

PitchShift solves this by designating the Left channel as the **Master**. The Left channel performs the heavy peak-detection and phase-shifting math. The Right channel acts as a **Slave**: it calculates the original phase difference ($\Delta\varphi_{stereo}$) between itself and the Left channel, and mathematically forces its output to match that exact difference, perfectly preserving the original stereo width.

Dynamic Makeup Gain

Due to the laws of Spectral Energy Spreading, stretching frequencies over a wider range (pitching up) causes a drop in audio amplitude. Combined with human psychoacoustics (the Fletcher-Munson curve), pitched-up audio sounds noticeably quiet.

To dynamically conserve acoustic power, PitchShift applies a continuous multiplier based on the pitch factor:

Gain Multiplier: $g = pf$ (if pitching up) | $g = \sqrt{pf}$ (if pitching down)

Formant Preservation

When a human sings, their vocal cords dictate the pitch, but their throat and mouth cavity (the vocal tract) dictate the resonance (Formants). Pitching a vocal up without correcting the formants shrinks the mathematical size of the throat, resulting in the "Chipmunk Effect."

When Formant Preservation is enabled, PitchShift extracts the Spectral Envelope (the general curve of the frequencies) of the original audio. After shifting the pitch, it applies a scaling matrix $(Env_{original} / Env_{shifted})$ to force the new pitch to adopt the original vocal tract resonance.

4. Music Information Retrieval (AI Key Detection)

PitchShift features a real-time Music Information Retrieval (MIR) system that mathematically determines the musical key of the playing track.

> **1. Chromagram Extraction & Filter**
>
> During the FFT process, the engine isolates the "musical midrange" (100Hz to 3000Hz). This acts as a Bandpass Filter, ignoring sub-bass (kick drums) and high-treble (cymbals). The remaining frequency magnitudes are compressed using $\sqrt{magnitude}$ to prevent loud peaks from hijacking the math. Finally, all frequencies are "folded" into 12 pitch classes (C, C#, D... B), forming a real-time Pitch Class Profile (Chroma).

> **2. The Krumhansl-Schmuckler Algorithm**
>
> The accumulated Chroma data is sent to the Main Thread via a `MessagePort`. The extension calculates the Pearson Correlation Coefficient between the live Chroma and mathematically ideal profiles for Major and Minor keys.

Pearson Correlation: $$r = \Sigma(x_i - \bar{x})(y_i - \bar{y}) \,/\, \sqrt{(\Sigma(x_i - \bar{x})^2 \, \Sigma(y_i - \bar{y})^2)}$$

Because pop songs feature passing chords and drum fills that can briefly confuse the algorithm, PitchShift employs a **Temporal Voting System**. The algorithm remembers the last 5 key guesses and requires a 60% majority vote (3 out of 5) before it officially locks in and updates the UI. This results in rock-solid, non-jittery key detection.

5. The User Interface & Integration

The extension communicates seamlessly between the isolated background processes and the user via:

- **Glassmorphic OSD (On-Screen Display):** A custom shadow-DOM element injected directly into the video player, styled with backdrop filters to provide immediate visual feedback without opening the popup.
- **Event Dispatching:** State variables (Semitones, Formant State, Detected Key) are constantly synced across the Popup UI and the Page using custom DOM events (`__pitchshift:state`).
- **SPA Resilience (Zombie-Proofing):** On sites like Spotify, audio elements are frequently detached from the DOM during track transitions. PitchShift monitors the `currentTime` heartbeat of the element. As long as the time is progressing, the engine maintains the connection, preventing the audio from crashing during playlist transitions.